COMPUTER NETWORKS - UE24CS252B
Lab 3: Exploring UDP with DNS using Wireshark

Description

In this experiment, students will use Wireshark to capture and analyze DNS queries and responses that use UDP. This will demonstrate the connectionless nature of UDP and how DNS efficiently resolves domain names to IP addresses.

Understanding DNS and UDP

What is DNS?

The **Domain Name System (DNS)** is a hierarchical naming system for computers, services, or resources connected to the Internet or a private network. It translates human-readable domain names (like www.google.com) into IP addresses (like 142.250.185.78) required for devices to communicate over the network.

How does DNS work?

1. **DNS Query Initiation:**

   When you type a domain name in your browser, your device sends a **DNS query** to resolve the name to an IP address.

2. **DNS Resolver:**

   A DNS resolver (usually operated by your ISP or a public DNS provider like Google or Cloudflare) forwards the query through the DNS hierarchy until it reaches an authoritative DNS server.

3. **DNS Response:**

   The authoritative DNS server responds with the corresponding IP address, which is sent back to the client.

DNS Query Types:

- **A (Address Record):** Maps a domain to an IPv4 address.
- **CNAME:** Maps a domain to another domain (alias).
- **AAAA:** Maps a domain to an IPv6 address.
- **MX:** Identifies mail servers for a domain.

Key DNS Components

1. **Recursive Resolver:** A DNS server that receives queries from client devices and resolves them by querying other DNS servers.
2. **Root DNS Servers:** The starting point of the DNS hierarchy, pointing to top-level domain (TLD) servers like .com, .org, .edu.
3. **Authoritative DNS Server:** Contains the actual records for a domain (e.g., A, CNAME).

DNS Query Flow

1. **Step 1:** The user's device sends a query to the local DNS resolver.
2. **Step 2:** If the resolver doesn't have the answer in its cache, it queries the root server.
3. **Step 3:** The root server refers the resolver to the appropriate TLD server (e.g., .com).
4. **Step 4:** The resolver queries the TLD server, which refers it to the authoritative DNS server.
5. **Step 5:** The authoritative server responds with the IP address, which is cached by the resolver and returned to the client.

Wireshark DNS Analysis Observations

1. **DNS Query:**
   - Source Port: Random high-numbered port (e.g., 54832).
   - Destination Port: 53.
   - Query Name: The domain being resolved.
2. **DNS Response:**
   - Source Port: 53.
   - Destination Port: Same as the query's source port.
   - Response Code: Indicates success or failure (e.g., NoError, NXDomain).
   - Answer Section: Contains resolved IP addresses.
3. **UDP Behavior in DNS:**
   - No retransmissions for lost packets.
   - Fast communication without setup delays.

Additional Notes for Students

1. **Understand DNS Logs:**
   - Tools like dig and nslookup generate and display detailed DNS query/response logs. These tools help simulate real-world DNS queries for analysis.
   -
2. **Practical DNS Use Cases:**
   - Load Balancing: DNS resolves the same domain to different IPs for distributing traffic.
   - CDN (Content Delivery Network): DNS resolves to the nearest server for faster delivery.

Steps to Follow: (Hands on)

1. **Launch Wireshark:**
   - Open Wireshark from the terminal:

- wireshark
- Wireshark will open a graphical interface.

2. **Select the Network Interface:**
    - In the Wireshark interface, you'll see a list of available network interfaces (e.g., eth0, wlan0).
    - Choose the active interface based on your connection type:
        - eth0 for wired connections.
        - wlan0 for wireless connections.
    - Click the interface name to begin live packet capturing.
3. **Start Capturing Packets:**
    - Click on the green **Start Capturing Packets** button (shaped like a shark fin) in the toolbar.
4. **Generate DNS Traffic Using the dig Command:**
    - Open a terminal and run a dig query to resolve a domain name. For example:
    - **dig www.google.com**
    - This sends a DNS query to the configured DNS server (usually your ISP or public DNS like Google or Cloudflare).
    - Run additional queries to generate more DNS traffic:
    - dig www.wikipedia.org
    - dig www.github.com
5. **Filter DNS Packets in Wireshark:**
    - Once packets are captured, you'll need to focus on DNS traffic.
    - In the **filter bar** at the top of Wireshark, enter the following filter:
    - **udp.port == 53**
    - This will display only UDP packets sent or received on port 53, which is the standard port for DNS traffic.
6. **Analyze DNS Query Packets:**
    - Locate a packet labeled **DNS Query** in the **Protocol** column.
    - Click on the packet to expand its details in the lower pane. Examine the following fields:
        - **Source IP and port:** The client (your computer) that sent the query.
        - **Destination IP and port:** The DNS server (e.g., Google DNS: 8.8.8.8) and port 53.

- **Query Name:** The domain name being resolved (e.g., www.google.com).

7. **Analyze DNS Response Packets:**
   - Locate the corresponding **DNS Response** packet.
   - Check for:
     - **Response Code:** Indicates whether the query was successful (NoError) or failed (e.g., NXDomain for a non-existent domain).
     - **Resolved IP Address:** The IP address returned for the queried domain name.
8. **Stop Capturing Packets:**
   - Click the red **Stop Capturing Packets** button in the toolbar.
   - Save the capture file if needed for later analysis.

```
adarshnema@adarshnema:~$ dig www.google.com

; <<>> DiG 9.18.28-0ubuntu0.24.04.1-Ubuntu <<>> www.google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39487
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;www.google.com.                        IN      A

;; ANSWER SECTION:
www.google.com.         42      IN      A       172.217.163.164

;; Query time: 14 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Jan 27 07:33:43 IST 2025
;; MSG SIZE  rcvd: 59
```

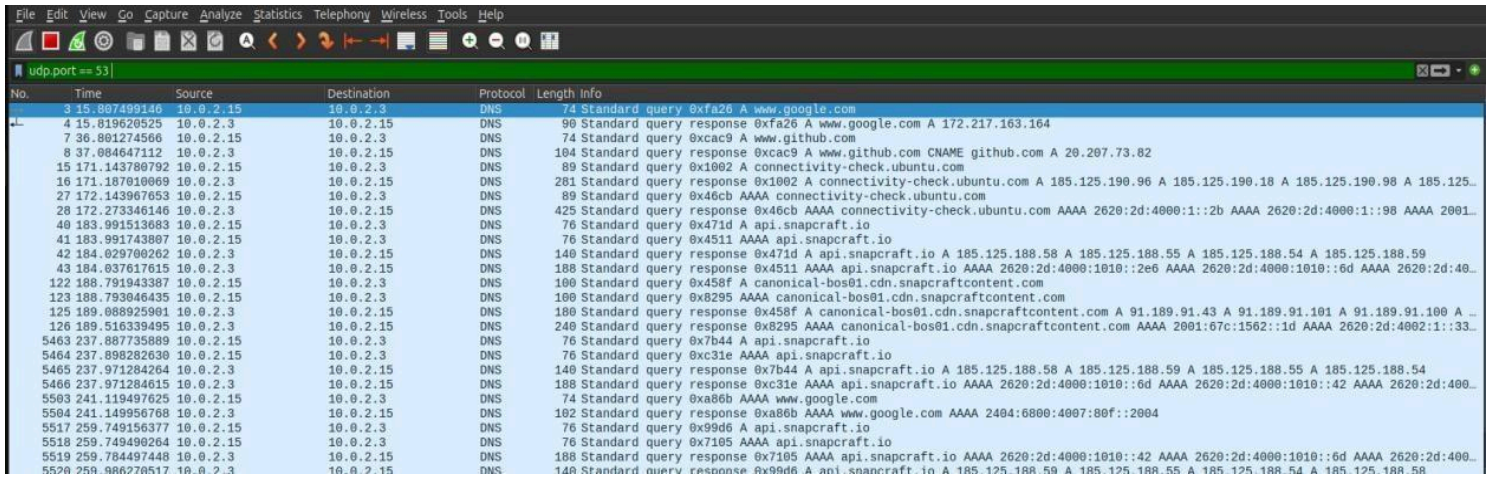
```
adarshnema@adarshnema:~$ dig www.google.com AAAA

; <<>> DiG 9.18.28-0ubuntu0.24.04.1-Ubuntu <<>> www.google.com AAAA
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 46521
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;www.google.com.                        IN      AAAA

;; ANSWER SECTION:
www.google.com.         183     IN      AAAA    2404:6800:4007:80f::2004

;; Query time: 36 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Jan 27 07:37:28 IST 2025
;; MSG SIZE  rcvd: 71
```



Analysis of dig www.google.com Using Wireshark

When you run the command dig www.google.com, the system initiates a DNS query to resolve the domain name to an IP address. Here's what happens:

Step-by-Step Breakdown

1. **Command Initiation:**
   - The dig command generates a DNS query for the A record of www.google.com.
   - The query is sent to the configured DNS resolver (local or public, like Google's 8.8.8.8).

2. **DNS Query in Wireshark:**
    - The first packet captured in Wireshark is:
    - Protocol: DNS
    - Info: Standard query 0xfa26 A www.google.com
        - **Protocol:** DNS indicates the packet is a DNS request.
        - **Query ID (0xfa26):** A unique identifier for the request, used to match the query with its response.
        - **Query Type (A):** Specifies the query is looking for an IPv4 address (A record).
        - **Query Name:** The domain name being resolved, www.google.com.
3. **DNS Response in Wireshark:**
    - Shortly after the query, the DNS resolver responds with the corresponding IP address:
    - Protocol: DNS
    - Info: Standard query response 0xfa26 A www.google.com A 172.217.163.164
        - **Query ID (0xfa26):** Matches the query, confirming this is the response to the initial request.
        - **Response Type (A):** Indicates that the resolver is providing an IPv4 address.
        - **Resolved IP (172.217.163.164):** The IP address of www.google.com.
4. **What You Observed:**
    - The query (Standard query 0xfa26 A www.google.com) uses **UDP** on port **53**.
    - The response (Standard query response 0xfa26 A www.google.com A 172.217.163.164) also uses **UDP** but includes the resolved IP address.

Key Points for Students to Note

- **Query ID:**
    - This unique identifier ensures that responses match their corresponding queries, even when multiple DNS queries are in transit.
- **Query Type:**
    - The **A record** requests the IPv4 address. Other query types include AAAA

(IPv6), MX (mail servers), and CNAME (aliases).

- **Response Contents:**
  - The response contains:
    1. **Answer Section:** Includes the resolved IP address.
    2. **Additional Information (if any):** May provide extra records (like CNAME or additional DNS server details).

Deep Dive Into Wireshark Fields

Field	Description
Transaction ID	Unique ID (0xfa26 in your example) linking the query and its response.
Flags	0x8180.
Questions	The domain name being queried (e.g., www.google.com).
Answers	Resolved IP address(es) (e.g., 172.217.163.164).
Query Type	Type of record requested (A, AAAA, CNAME, etc.).
Response Time	Measure of how quickly the DNS resolver responded.

Common Variations in DNS Responses

Authoritative Response:

If the DNS server is authoritative, the response flags will indicate so.

Nonexistent Domain (NXDomain):

If the domain doesn't exist, the response will include an error code (NXDomain).

Multiple IPs:

For load balancing, a single domain may resolve to multiple IPs, which appear in the **Answer Section**.

Deliverables:

Part 1: DNS Analysis

1. **Observe DNS Caching:**

- Run `dig www.google.com` twice and observe the difference. The second query may be resolved locally due to caching, resulting in a faster response.
- Screenshot of Wireshark output for dig command.

2. **Query for Other Record Types:**
   - Try dig www.google.com AAAA to observe the IPv6 address.
   - Use dig google.com MX to view mail server records.
   - Screenshot of Wireshark and terminal output
3. **Modify DNS Servers:**
   - Change the DNS server to a public one like 1.1.1.1 (Cloudflare) or 8.8.8.8 (Google).
   - Compare the response times and observe differences in the DNS resolution process.
   - Screenshots of Wireshark and terminal output
4. **Analyze DNS over TCP:**
   - Use **dig +tcp www.google.com** to force DNS over TCP.
   - Capture and compare the packet behavior in Wireshark. Explain, what do you observe along with Screenshot capture.

Part 2: UDP Communication

1. Set up a UDP server using **netcat** on port 54321.
2. Send three distinct messages from a client and capture the traffic with Wireshark. Screenshot of the traffic capture
3. Write a report:
   - How does UDP differ from TCP?
   - Why is UDP suitable for DNS?
   - Observations about UDP traffic in Wireshark.

Part 3: Analysis of the UDP protocol in DNS using Wireshark (Screenshots and Explanations are required)

1. Select the first UDP segment in your trace.
   1. What is the packet number of this segment in the trace file?
   2. What type of application-layer payload or protocol message is being carried in this UDP segment? Look at the details of this packet in Wireshark.
   3. How many fields are there in the UDP header? What are the names of these fields?

2. By consulting the displayed information in Wireshark's packet content field for this packet, what is the length (in bytes) of the UDP header field?

3. Examine the pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet.
    A. What is the packet number of these two UDP segments in the trace file?
    B. What is the value in the source port field in this UDP segment?
    C. What is the value in the destination port field in this UDP segment?
    D. What is the value in the source port field in this second UDP segment?
    E. What is the value in the destination port field in this second UDP segment? Describe the relationship between the port numbers in the two packets.

Example:

